Catalog Microservice

Code Explanation + Business Logic Documentation

C# .NET | Clean Architecture | CQRS | MediatR Pattern

Catalog Microservice -

Catalog ( ) microservice .

- : Pagination
- : ID
- :
- :

MongoDB (NoSQL) :

- (attributes)
- Regex Search

(Architecture)

```
Catalog Service
|-- Catalog.API             <-   - Controllers - HTTP Endpoints
|-- Catalog.Application     <-   - Commands/Queries/Handlers/Mappers
|-- Catalog.Core            <-  - Entities - Interfaces - Specification
\-- Catalog.Infrastructure  <-  - MongoDB Repositories - Data Seeding
```

Clean Architecture + CQRS + Mediator Pattern .

Catalog.Core -

>> Entities ()

BaseEntity.cs

```
public class BaseEntity
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; }
```

```
}
```

: .

- [[BsonId]]: MongoDB Primary Key
- [[BsonRepresentation(BsonType.ObjectId)]]: ObjectId MongoDB string C#

Product.cs

```
public class Product : BaseEntity
{
    public string Name { get; set; }
    public string Summary { get; set; }
    public string Description { get; set; }
    public string ImageFile { get; set; }
    public ProductBrand Brand { get; set; }
    public ProductType Type { get; set; }

    [BsonRepresentation(BsonType.Decimal128)]
    public decimal Price { get; set; }
    public DateTimeOffset CreatedDate { get; set; }
}
```

: .

- [Name, Summary, Description]:
- [Brand]: (Nike, Adidas, etc.) - Embedded Document
- [Type]: (Shoes, Clothes, etc.) - Embedded Document
- [[BsonRepresentation(BsonType.Decimal128)]]: MongoDB
- [CreatedDate]:

ProductBrand.cs

```
public class ProductBrand : BaseEntity
{
    [BsonElement("Name")]
    public string Name { get; set; }
}
```

: (: Nike, Adidas, Apple).

- [[BsonElement("Name")]]: Field MongoDB Document

ProductType.cs

```
public class ProductType : BaseEntity
{
    public string Name { get; set; }
}
```

: (: Shoes, Electronics, Clothes).

>> Specification ( )

CatalogSpecParams.cs

```
public class CatalogSpecParams
{
    public int MaxPageSize { get; set; } = 70;
    public int _PageSize { get; set; } = 10;
    public int PageIndex { get; set; } = 1;

    public int PageSize
    {
        get => _PageSize;
        set => _PageSize = (value > MaxPageSize) ? MaxPageSize : value;
    }

    public string? BrandId { get; set; }
    public string? TypeId { get; set; }
    public string? Sort { get; set; }
    public string? Search { get; set; }
}
```

: Client:

- [PageIndex, PageSize]: Pagination ( )
- [MaxPageSize = 70]: Server
- [BrandId]:
- [TypeId]:
- [Sort]: (priceAsc, priceDesc, nameAsc)
- [Search]:

Pagination.cs

```
public class Pagination<T> where T : class
{
    public int PageIndex { get; set; } = 1;
    public int PageSize { get; set; }
    public int Count { get; set; }
    public IReadOnlyCollection<T> Data { get; set; }
}
```

: Generic class Pagination.

- [Count]: (Frontend)
- [Data]:
- [T] Product Entity

>> Repository Interfaces

IProductRepository.cs

```
public interface IProductRepository
{
    Task<IEnumerable<Product>> GetAllAsync();
```

```
    Task<Pagination<Product>> GetProducts(CatalogSpecParams catalogSpecParams);
    Task<IEnumerable<Product>> GetProductByName(string Name);
    Task<IEnumerable<Product>> GetProductByBrand(string Name);
    Task<Product> GetProduct(string productId);
    Task<Product> CreateProduct(Product product);
    Task<bool> UpdateProduct(Product product);
    Task<bool> DeleteProduct(string productId);
    Task<ProductBrand> GetBrandByIdAsync(string brandId);
    Task<ProductType> GetTypeByIdAsync(string typeId);
}
```

: :

- [GetProducts]: Pagination
- [GetProductByName]: ( )
- [UpdateProduct]: bool (true = , false =)
- [DeleteProduct]: bool (true = , false =)

Catalog.Infrastructure -

>> ProductRepository.cs

```
public class ProductRepository : IProductRepository
{
    private readonly IMongoCollection<ProductBrand> _brands;
    private readonly IMongoCollection<ProductType> _types;
    private readonly IMongoCollection<Product> _products;

    public ProductRepository(IOptions<DatabaseSettings> options)
    {
        var settings = options.Value;
        var client = new MongoClient(settings.ConnectionString);
        var db = client.GetDatabase(settings.DatabaseName);
        _brands = db.GetCollection<ProductBrand>(settings.BrandCollectionName);
        _types = db.GetCollection<ProductType>(settings.TypeCollectionName);
        _products = db.GetCollection<Product>(settings.ProductCollectionName);
    }
    ...
}
```

: MongoDB Collections .

CreateProduct

```
public async Task<Product> CreateProduct(Product product)
{
    await _products.InsertOneAsync(product);
    return product;
}
```

MongoDB (ID).

DeleteProduct

```
public async Task<bool> DeleteProduct(string productId)
{
    var deletedProduct = await _products.DeleteOneAsync(p => p.Id == productId);
    return deletedProduct.IsAcknowledged && deletedProduct.DeletedCount > 0;
}
```

- [IsAcknowledged]: MongoDB
- [DeletedCount > 0]: (ID)

GetProductByName (Regex)

```
public async Task<IEnumerable<Product>> GetProductByName(string Name)
{
    var filter = Builders<Product>.Filter.Regex(
        p => p.Name,
        new BsonRegularExpression($".*{Name}.*", "i")
    );
    return await _products.Find(filter).ToListAsync();
}
```

- Regex: [.*Nike.*] "Nike Air" "Blue Nike"
- [i] = Case Insensitive (/)

GetProducts (Pagination)

```
public async Task<Pagination<Product>> GetProducts(CatalogSpecParams catalogSpecParams)
{
    var builder = Builders<Product>.Filter;
    var filter = builder.Empty;

    //
    if (!string.IsNullOrEmpty(catalogSpecParams.Search))
        filter &= builder.Where(p => p.Name.ToLower().Contains(catalogSpecParams.Search.ToLower()));

    //
    if (!string.IsNullOrEmpty(catalogSpecParams.BrandId))
        filter &= builder.Eq(p => p.Brand.Id, catalogSpecParams.BrandId);

    //
    if (!string.IsNullOrEmpty(catalogSpecParams.TypeId))
        filter &= builder.Eq(p => p.Type.Id, catalogSpecParams.TypeId);

    var totalItems = await _products.CountDocumentsAsync(filter);
    var data = await ApplyDataFilters(catalogSpecParams, filter);

    return new Pagination<Product>
    {
        PageIndex = catalogSpecParams.PageIndex,
        PageSize = catalogSpecParams.PageSize,
        Count = (int)totalItems,
        Data = data
    };
```

```
}
```

- Filter Client
- [filter &=]: (AND)
- Pagination Frontend

Pagination

```
private async Task<IReadOnlyCollection<Product>> ApplyDataFilters(
    CatalogSpecParams catalogSpecParams,
    FilterDefinition<Product> filter)
{
    var sortDefn = Builders<Product>.Sort.Ascending("Name");

    if (!string.IsNullOrEmpty(catalogSpecParams.Sort))
    {
        sortDefn = catalogSpecParams.Sort switch
        {
            "priceAsc"  => Builders<Product>.Sort.Ascending(p => p.Price),
            "priceDesc" => Builders<Product>.Sort.Descending(p => p.Price),
            _           => Builders<Product>.Sort.Ascending(p => p.Name)
        };
    }

    return await _products
        .Find(filter)
        .Sort(sortDefn)
        .Skip(catalogSpecParams.PageSize * (catalogSpecParams.PageIndex - 1))
        .Limit(catalogSpecParams.PageSize)
        .ToListAsync();
}
```

- :
- [priceAsc]:
- [priceDesc]:
- [Skip]:
- [Limit]:

>> DatabaseSeeder.cs -

```
public class DatabaseSeeder
{
    public static async Task SeedAsync(IOptions<DatabaseSettings> options)
    {
        //   MongoDB
        //   Brands  ->  brands.json
        //   Products  ->  products.json
        //   Product.Id = null  MongoDB ID
        //  CreatedDate
    }
}
```

: MongoDB . .

Catalog.Application -

>> Commands ()

CreateProductCommand.cs

```
public record CreateProductCommand(
    string Name,
    string Summary,
    string Description,
    string ImageFile,
    string BrandId,
    string TypeId,
    decimal Price
) : IRequest<ProductResponse>;
```

: API.

UpdateProductCommand.cs

```
public record UpdateProductCommand(
    string Id,
    string Name,
    string Summary,
    string Description,
    string ImageFile,
    string BrandId,
    string TypeId,
    decimal Price
) : IRequest<bool>;
```

: . [bool] Controller .

DeleteProductCommand.cs

```
public record DeleteProductCommand(string Id) : IRequest<bool>;
```

: . ID .

>> Queries ()

Query	
`GetAllProductsQuery(C	Pagination
`GetProuctByIdQuery(strin	ID
`GetProductByNameQuery(st	
`GetAllProductByBrandQuer	

`GetAllBrandsQuery()`	
`GetAllTypesQuery()`	

>> Handlers ()

CreateProductCommandHandler.cs

```
public async Task<ProductResponse> Handle(CreateProductCommand request, CancellationToken ct)
{
    // 1.  Brand  MongoDB  BrandId
    var brand = await _productRepository.GetBrandByIdAsync(request.BrandId);

    // 2.  Type  MongoDB  TypeId
    var type = await _productRepository.GetTypeByIdAsync(request.TypeId);

    // 3.  Product Entity
    var product = new Product
    {
        Name = request.Name,
        Brand = brand,
        Type = type,
        Price = request.Price,
        ...
        CreatedDate = DateTime.UtcNow
    };

    // 4.   MongoDB
    var createdProduct = await _productRepository.CreateProduct(product);

    // 5.   Response
    return createdProduct.ToResponse();
}
```

1. Brand Type MongoDB
2. Product Entity Brand Type Embedded Documents
3. MongoDB Response

UpdateProductCommandHandler.cs

```
public async Task<bool> Handle(UpdateProductCommand request, CancellationToken ct)
{
    // 1.
    var existingProduct = await _productRepository.GetProduct(request.Id);
    if (existingProduct == null) return false;

    // 2.  Brand Type
    var brand = await _productRepository.GetBrandByIdAsync(request.BrandId);
    var type = await _productRepository.GetTypeByIdAsync(request.TypeId);

    // 3.
    existingProduct.Name = request.Name;
    existingProduct.Brand = brand;
```

```
    existingProduct.Type = type;
    existingProduct.Price = request.Price;
    ...

    // 4.
    return await _productRepository.UpdateProduct(existingProduct);
}
```

2. Embedded Documents

GetAllProductsHandler.cs

```
public async Task<IList<ProductResponse>> Handle(GetAllProductsQuery request, CancellationToken ct)
{
    var products = await _productRepository.GetProducts(request.CatalogSpecParams);
    return products.Data.Select(p => p.ToResponse()).ToList();
}
```

: Repository Responses.

Catalog.API -

>> CatalogController.cs - API Endpoints

```
[ApiController]
[Route("api/v1/[controller]")]
public class CatalogController : ControllerBase
{
    private readonly IMediator _mediator;
    ...
}
```

HTTP Method	URL		Handler
GET	**/api/v1/catalog/GetAll**		**GetAllProductsHandler**
GET	/api/v1/catalog/{id}	ID	GetProductByIdHandler
GET	/api/v1/catalog/productNa		GetProductByNameHandler
GET	/brand/{brand}		GetAllProductByBrandHandl
GET	/api/v1/catalog/GetAllBra		GetAllBrandsHandler
GET	/api/v1/catalog/GetAllTyp		GetAllTypesHandler
POST	/api/v1/catalog		CreateProductCommandHandl
PUT	/api/v1/catalog/{id}		UpdateProductCommandHandl
DELETE	/api/v1/catalog/{id}		DeleteProductCommandHandl

- GetAllProducts

```
[HttpGet("GetAllProducts")]
public async Task<ActionResult<IList<ProductDto>>> GetAllProducts(
    [FromQuery] CatalogSpecParams catalogSpecParams)
```

```
{
    var query = new GetAllProductsQuery(catalogSpecParams);
    var result = await _mediator.Send(query);
    return Ok(result);
}
```

: [[FromQuery]] Query Parameters CatalogSpecParams object.

- GetProductByProductName

```
[HttpGet("productName/{productName}")]
public async Task<ActionResult<IList<ProductDto>>> GetProductByProductName(string productName)
{
    var query = new GetProductByNameQuery(productName);
    var result = await _mediator.Send(query);

    if (result == null || !result.Any())
        return NotFound();

    var dtoList = result.Select(p => p.ToDto()).ToList();
    return Ok(dtoList);
}
```

: 404 NotFound .

>> Program.cs -

```
//  MongoDB Serializers
BsonSerializer.RegisterSerializer(new GuidSerializer(BsonType.String));
BsonSerializer.RegisterSerializer(new DateTimeOffsetSerializer(BsonType.String));

//   Repositories
builder.Services.AddScoped<IBrandRepository, BrandRepository>();
builder.Services.AddScoped<ITypeRepository, TypeRepository>();
builder.Services.AddScoped<IProductRepository, ProductRepository>();

//  MongoDB
builder.Services.Configure<DatabaseSettings>(
    builder.Configuration.GetSection("DatabaseSettings"));

// MongoClient  Singleton (     Requests)
builder.Services.AddSingleton<IMongoClient>(sp =>
{
    var settings = sp.GetRequiredService<IOptions<DatabaseSettings>>().Value;
    return new MongoClient(settings.ConnectionString);
});

// Seed    Startup
using (var scope = app.Services.CreateScope())
{
    var options = scope.ServiceProvider.GetRequiredService<IOptions<DatabaseSettings>>();
    await DatabaseSeeder.SeedAsync(options);
}
```

- MongoClient Singleton
  - MongoDB
  - [MongoClient] Thread-Safe Requests
  - instance

(Data Flow)

>> (POST)

```
Client (Admin Panel)
  | POST /api/v1/catalog
  | Body: { name, brandId, typeId, price, ... }

CatalogController.CreateProduct()
  |  CreateProductCommand  MediatR

CreateProductCommandHandler.Handle()
  | GetBrandByIdAsync(brandId)   -> MongoDB
  | GetTypeByIdAsync(typeId)     -> MongoDB
  |  Product Entity  Embedded Brand  Type
  | CreateProduct(entity)        -> MongoDB InsertOne

ProductRepository.CreateProduct()
  | _products.InsertOneAsync(product)
  | MongoDB  ObjectId
  |   Product   ID

CreateProductCommandHandler
  | product.ToResponse()

Client <- ProductResponse (  ID )
```

>> (GET)

```
Client (Frontend)
  | GET /api/v1/catalog/GetAllProducts
  |       ?search=nike&brandId=abc&sort=priceAsc&pageIndex=2&pageSize=10

CatalogController.GetAllProducts(CatalogSpecParams)
  | CatalogSpecParams     Query String
  |  GetAllProductsQuery(specParams)

GetAllProductsHandler.Handle()
  | _productRepository.GetProducts(specParams)

ProductRepository.GetProducts()
  |  MongoDB Filter:
  |   WHERE Name LIKE '%nike%'
  |     AND Brand.Id = 'abc'
  |  Total Count
  |  Sort: Price ASC
```

```
  |  Pagination: Skip 10, Limit 10

MongoDB  10

Pagination<Product> { Count: 45, Data: [10 products] }

GetAllProductsHandler
  |  product.ToResponse()

Client <- IList<ProductResponse>
  ( Count=45   )
```

MediatR	CQRS Mediator Pattern
MongoDB.Driver	MongoDB
MongoDB.Bson	C# Objects BSON
System.Text.Json	Seed JSON Files

Catalog Basket Services

	Basket Service	Catalog Service
** **	Redis (Cache)	MongoDB (Document DB)
** **		
** CRUD**	3	(9)
****		+ + +
** Pagination**		
** **		DatabaseSeeder
** Entities**		(Embedded Docs)

Catalog :

1. :
2. :
3. Basket:

REST API Frontend .